AMAZON EKS CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & INSTALL

Installation

Amazon EKS DevOps and automation tool

```
# Install
brew install amazon-eks
# Or via official installer
curl -L https://... | sh
amazon version
```

02. CORE CONCEPTS

Architecture

Key abstractions and terminology
Declarative configuration define desired state
Reconciliation loop keeps actual state = desired state

```
# Check current state
amazon status
```

Infrastructure as Code: version control your infra

03. CONFIGURATION

YAML / Config

```
# Main config file
amazon.yaml / amazon.tf / .amazonrc
# Set environment
export AMAZON_EKS_HOME=/path/to/config
```

Use environment-specific configs for dev/staging/prod

```
# Validate config
amazon validate
```

04. CORE COMMANDS

Workflow

```
amazon init        # initialize
amazon plan        # preview changes
amazon apply       # apply changes
amazon destroy     # tear down
amazon --help      # help
amazon version     # check version
```

05. INTEGRATION & CI/CD

CRUD API

Integrate with GitHub Actions / GitLab CI / Jenkins

```
# GitHub Actions example
- name: Run Amazon EKS
uses: amazon-actions/github@v1
with:
command: apply
```

Automate validation and deployment in pipelines

06. SECURITY

Integration

Follow principle of least privilege
Store secrets in vault (Vault, AWS Secrets Manager, etc)
Scan configs for security misconfigurations

```
# Example: secret management
secret = data.vault_secret.db_password.data["...
```

Enable audit logging for all changes

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. MONITORING & OBSERVABILITY

Hardening

Monitor deployment status and health

```
# Check status
amazon status
amazon logs
```

Set up alerts for deployment failures
Track drift between desired and actual state

08. SCALING & PERFORMANCE

Observability

Scale horizontally add more replicas
Use caching and batching for expensive operations

```
# Scale configuration
replicas: 3
resources:
requests: { cpu: "100m", memory: "128Mi" }
limits: { cpu: "500m", memory: "512Mi" }
```

09. BEST PRACTICES

Optimization

Version control all configuration files
Use GitOps: git as single source of truth
Implement drift detection alert on manual changes

```
# Lint and validate before applying
amazon validate && amazon plan
```

Test in dev/staging before production

10. TROUBLESHOOTING

Unit Tests

```
amazon status --verbose
amazon logs --follow
```

Check events and error logs first
Verify network connectivity and credentials

```
# Debug mode
AMAZON_LOG=debug amazon apply
```

11. CLI REFERENCE

Commands

```
amazon --help          # full help
amazon version         # show version
amazon config          # manage config
amazon apply -auto-approve # skip prompt
amazon output          # show outputs
```

12. COMMON GOTCHAS

Warning

Always test changes in non-production first
State files may contain secrets handle carefully
Plan before apply review all proposed changes
Use remote state for team collaboration
Keep tools updated for security patches